
인공지능을 위한 선형대수학

실습 1. NumPy로 만나는 벡터와 행렬

Google Colab 시작하기

설치할 것이 없습니다

- 1 브라우저에서 colab.research.google.com 접속
- 2 구글 계정으로 로그인
- 3 선생님이 공유한 내 이름의 노트북 링크 열기
사본을 만들지 마세요 — 그 파일에 바로 쓰면 자동 저장됩니다.
- 4 코드 칸을 클릭하고 **Shift + Enter** → 실행!



Colab 화면 익히기

셀(cell)이 기본 단위

- **코드 셀** — 파이썬 코드를 쓰는 칸. 왼쪽 ► 버튼이나 Shift + Enter 로 실행
- **텍스트 셀** — 설명이 적힌 칸. 실행하지 않습니다
- 실행하면 셀 **바로 아래**에 결과가 나타납니다
- 셀 왼쪽의 [1], [2] 는 **실행한 순서**입니다



SECTION 1

준비

NumPy 불러오기



NumPy 불러오기

Importing NumPy

[1]

따라 쓰기

```
import numpy as np  
print(np.__version__)
```

실행 결과

2.0.2

SECTION 2

벡터 만들기

Creating Vectors

벡터와 크기 확인

Vector and shape

[2]

따라 쓰기

```
# 벡터 만들기  
c = np.array([1, 2, 3])  
print(c)  
print(c.shape)
```

실행 결과

[1 2 3]

(3,)

성분 꺼내기

Indexing — 0부터 셉니다

[3]

따라 쓰기

```
# 번호는 0부터 시작!  
print(c[0])  
print(c[2])
```

실행 결과

1

3

행벡터 만들기

Row Vector — 대괄호 한 겹 더

[4]

따라 쓰기

```
# 행벡터 (1행 3열)
r = np.array([[1, 2, 3]])
print(r.shape)
print(r[0, 1])
```

실행 결과

(1, 3)

2

SECTION 3

행렬과 전치

Matrix and Transpose

행렬 만들기

안쪽 대괄호 하나가 한 줄(행)

[5]

따라 쓰기

```
A = np.array([[1, 2],  
              [3, 4],  
              [5, 6]])  
  
print(A)  
print(A.shape)
```

실행 결과

```
[[1 2]  
 [3 4]  
 [5 6]]  
(3, 2)
```

전치

Transpose — 점 T 하나면 끝

[6]

따라 쓰기

```
print(A.T)
print(A.T.shape)
```

실행 결과

```
[[1 3 5]
 [2 4 6]]
(2, 3)
```

자주 쓰는 행렬 빠르게 만들기

zeros · ones · full · random

[7]

따라 쓰기

```
print(np.zeros((2, 2)))      # 모두 0
print(np.ones((2, 2)))       # 모두 1
print(np.full((2, 2), 7))    # 모두 7
```

실행 결과

```
[[0. 0.]
 [0. 0.]]
[[1. 1.]
 [1. 1.]]
[[7 7]
 [7 7]]
```


SECTION 4

행렬 곱셈

Matrix Multiplication

행렬 곱셈은 @

Matrix Multiplication with @

[8]

따라 쓰기

```
B = np.array([[11, 12, 13, 14],
               [15, 16, 17, 18]])
print(A.shape, B.shape)
print(A @ B)
```

실행 결과

```
(3, 2) (2, 4)
[[ 41  44  47  50]
 [ 93 100 107 114]
 [145 156 167 178]]
```

크기가 안 맞으면?

일부러 에러를 내 보시다

[9]

따라 쓰기

```
# B는 (2, 4), A는 (3, 2) → 4 ≠ 3
```

```
print(B @ A)
```


* 와 @ 는 완전히 다르다

Element-wise vs Matrix Product

[10]

따라 쓰기

```
P = np.array([[1, 2], [3, 4]])
Q = np.array([[5, 6], [7, 8]])
print(P * Q)    # 같은 자리끼리
print(P @ Q)    # 진짜 행렬 곱
```

실행 결과

```
[[ 5 12]
 [21 32]]
[[19 22]
 [43 50]]
```

AB ≠ BA 를 코드로 확인

Not Commutative

[11]

따라 쓰기

```
print(P @ Q)
print(Q @ P)
print(np.array_equal(P @ Q, Q @ P))
```

실행 결과

```
[[19 22]
 [43 50]]
[[23 34]
 [31 46]]
False
```

직접 해보기

- 1 다음 행렬을 만들고 크기와 전치를 출력하시오.
`M = [[2, 0, 1], [4, 5, 3]]`
- 2 아래 두 행렬에 대해 $X @ Y$ 와 $Y @ X$ 를 각각 계산하고, 결과가 다름을 확인하시오.
`X = [[2, 0], [1, 3]]      Y = [[4, -1], [2, 5]]`
- 3 A 가 (3, 5), B 가 (5, 2)일 때 $(A @ B).T$ 와 $B.T @ A.T$ 의 **shape가 같음**을 코드로 확인하시오.
힌트 - `np.zeros((3, 5))` 처럼 아무 값으로 만들어도 크기 확인에는 충분

오늘 배운 명령어

Summary

코드	뜻
<code>np.array([...])</code>	벡터.행렬 만들기
<code>.shape</code>	크기 확인 → (행, 열)
<code>.T</code>	전치. 크기도 뒤집힘
<code>A @ B</code>	행렬 곱셈 — 가운데 숫자가 맞아야 함
<code>A * B</code>	성분별 곱 — 행렬 곱이 아님!
<code>np.zeros / ones / full</code>	특정 값으로 채운 행렬
<code>np.array_equal(A, B)</code>	두 행렬이 같은지 True / False